10 — System Catalogs

> ***Module:*** *PostgreSQL Internals* | ***Difficulty:*** *Advanced* | ***Est. reading time:*** *60 min*

Table of Contents

1. Learning Objectives
2. Overview
3. Key Catalogs
   - pg_class
   - pg_attribute
   - pg_index
   - pg_statistic / pg_stats
   - pg_proc
   - pg_roles / pg_user
4. Key Views
   - pg_stat_activity
   - pg_stat_user_tables
   - pg_stat_user_indexes
   - pg_locks
   - pg_replication_slots
5. Catalog Query Patterns
6. 20+ Useful Catalog Queries
7. ASCII Diagram: Catalog Relationships
8. Common Mistakes
9. Best Practices
10. Performance Considerations
11. Interview Questions
12. Exercises with Solutions
13. Cross-References

Learning Objectives

After completing this section you will be able to:

- Navigate the PostgreSQL system catalog confidently, knowing which table to query for each type of metadata.
- Write queries against `pg_class`, `pg_attribute`, `pg_index`, and `pg_statistic` to answer structural questions.
- Use monitoring views (`pg_stat_activity`, `pg_stat_user_tables`, `pg_locks`) for live diagnostics.
- Distinguish catalog tables (always present) from statistics views (updated by the stats collector asynchronously).
- Avoid common catalog query pitfalls (wrong schema filter, missing dead-tuple handling, etc.).

Overview

PostgreSQL stores all metadata about the database — table definitions, column types, index information, function signatures, roles, statistics — in a set of **system catalogs** stored in the `pg_catalog` schema. These are regular tables and views; you can query them with normal SQL.

Categories:

- **Schema catalogs** — structural metadata (tables, columns, indexes, types, functions).
- **Statistics catalogs** — collected by the stats collector (`pg_statistic` , `pg_stats`).
- **Activity views** — real-time server state (connections, locks, replication, vacuum progress).

Key Catalogs

pg_class

Every **relation** in PostgreSQL — tables, indexes, sequences, views, materialized views, composite types, TOAST tables — has a row in `pg_class` .

Key Column	Type	Description
`oid`	oid	Object identifier
`relname`	name	Relation name
`relnamespace`	oid	Schema OID (references `pg_namespace.oid`)
`reltype`	oid	Type OID (for composite types)
`relowner`	oid	Owner role OID
`relam`	oid	Access method OID (for indexes: btree, hash, gin…)
`relfilenode`	oid	Physical file name (differs from oid after rewrite)
`reltablespace`	oid	Tablespace OID (0 = default)
`relpages`	int4	Estimated number of pages
`reltuples`	float4	Estimated total tuple count
`relallvisible`	int4	Number of all-visible pages (from VM)
`reltoastrelid`	oid	TOAST table OID (0 if none)
`relhasindex`	bool	Has at least one index
`relkind`	char	Relation kind: r=table, i=index, S=sequence, v=view, m=materialized view, c=composite type, t=TOAST, p=partitioned table, f=foreign table
`relfrozenxid`	xid	Oldest XID still present in the table (for wraparound tracking)
`reloptions`	text[]	Storage parameters (fillfactor, autovacuum_*, etc.)

```
SELECT oid, relname, relkind, reltuples::bigint, relpages,
       pg_size_pretty(pg_relation_size(oid)) AS size
FROM pg_class
WHERE relnamespace = (SELECT oid FROM pg_namespace WHERE nspname = 'public')
```

```
  AND relkind = 'r'
ORDER BY relpages DESC;
```

---

pg_attribute

Every **column** in every relation has a row in `pg_attribute`.

Key Column	Type	Description
`attrelid`	oid	Table OID
`attname`	name	Column name
`atttypid`	oid	Data type OID
`attnum`	int2	Column position (1-based; negative = system column)
`attlen`	int2	Fixed-length type size; -1 = variable; -2 = cstring
`atttypmod`	int4	Type-specific modifier (e.g., varchar(100) → 100+4)
`attnotnull`	bool	NOT NULL constraint
`atthasdef`	bool	Column has a default value
`attisdropped`	bool	Column was dropped (ghost column)
`attstorage`	char	TOAST strategy: p/e/m/x
`attstattarget`	int4	Statistics collection target (-1 = global default)
`attcompression`	char	Compression algorithm: `\0`=default, `p`=pglz, `l`=lz4

```
SELECT a.attname, t.typname, a.attnotnull,
       CASE a.attstorage
           WHEN 'p' THEN 'PLAIN' WHEN 'e' THEN 'EXTERNAL'
           WHEN 'm' THEN 'MAIN'  WHEN 'x' THEN 'EXTENDED'
       END AS toast_strategy,
       a.attstattarget
FROM pg_attribute a
JOIN pg_type t ON t.oid = a.atttypid
WHERE a.attrelid = 'orders'::regclass
  AND a.attnum > 0
  AND NOT a.attisdropped
ORDER BY a.attnum;
```

---

pg_index

Every **index** has a row in `pg_index` (in addition to its `pg_class` row).

Key Column	Type	Description
`indexrelid`	oid	OID of the index in pg_class

`indrelid`	oid	OID of the table being indexed
`indnatts`	int2	Number of index key columns
`indnkeyatts`	int2	Number of key columns (excludes INCLUDE columns)
`indisunique`	bool	Is the index unique?
`indisprimary`	bool	Is it the primary key?
`indisexclusion`	bool	Is it an exclusion constraint?
`indimmediate`	bool	Is the uniqueness check immediate?
`indisclustered`	bool	Last CLUSTER used this index?
`indisvalid`	bool	Is the index currently valid (not being built)?
`indisready`	bool	Is the index ready for inserts?
`indislive`	bool	Is the index live (not dropped)?
`indkey`	int2vector	Array of column numbers indexed
`indpred`	pg_node_tree	Partial index predicate (NULL if not partial)
`indexprs`	pg_node_tree	Expression index columns (NULL if not expression index)

```
SELECT
    i.relname              AS index_name,
    ix.indisprimary,
    ix.indisunique,
    array_agg(a.attname ORDER BY k.pos) AS indexed_columns,
    ix.indpred IS NOT NULL AS is_partial,
    ix.indexprs IS NOT NULL AS is_expression
FROM pg_index ix
JOIN pg_class i  ON i.oid  = ix.indexrelid
JOIN pg_class t  ON t.oid  = ix.indrelid
JOIN pg_attribute a ON a.attrelid = t.oid
CROSS JOIN LATERAL unnest(ix.indkey) WITH ORDINALITY AS k(col, pos)
WHERE a.attnum = k.col
  AND t.relname = 'orders'
  AND t.relnamespace = (SELECT oid FROM pg_namespace WHERE nspname = 'public')
GROUP BY i.relname, ix.indisprimary, ix.indisunique, ix.indpred, ix.indexprs
ORDER BY index_name;
```

---

pg_statistic / pg_stats

`pg_statistic` stores per-column statistics used by the query planner. It is updated by `ANALYZE`.
`pg_stats` is a user-friendly view over `pg_statistic`.

Key Column in pg_stats	Description

`tablename`	Table name
`attname`	Column name
`null_frac`	Fraction of entries that are NULL
`avg_width`	Average byte width of non-NULL entries
`n_distinct`	Distinct value count (negative = fraction of rows)
`most_common_vals`	Array of most common values
`most_common_freqs`	Frequencies matching most_common_vals
`histogram_bounds`	Histogram bucket boundaries
`correlation`	Statistical correlation between physical and logical order (-1 to 1)
`most_common_elems`	For array columns: most common array elements

```
SELECT attname, null_frac, avg_width, n_distinct,
       left(most_common_vals::text, 60) AS top_vals,
       left(histogram_bounds::text, 60) AS histogram,
       correlation
FROM pg_stats
WHERE tablename = 'orders'
  AND schemaname = 'public'
ORDER BY attname;
```

---

pg_proc

Every **function and procedure** has a row in `pg_proc`.

Key Column	Description
`proname`	Function name
`pronamespace`	Schema OID
`proowner`	Owner role OID
`prolang`	Language OID (sql, plpgsql, c, ...)
`prorettype`	Return type OID
`proargtypes`	Argument type OIDs
`prosrc`	Source code text (for interpreted languages)
`provolatile`	'i'=immutable, 's'=stable, 'v'=volatile
`proisstrict`	Returns NULL if any argument is NULL
`prosecdef`	SECURITY DEFINER

`proparallel`	'u'=unsafe, 'r'=restricted, 's'=safe

```
SELECT proname, prolang::regprocedure,
       pg_get_function_arguments(oid) AS args,
       pg_get_function_result(oid)    AS returns,
       provolatile, proparallel
FROM pg_proc
WHERE pronamespace = (SELECT oid FROM pg_namespace WHERE nspname = 'public')
ORDER BY proname;
```

---

pg_roles / pg_user

`pg_roles` is a cluster-wide catalog (in `global/`) listing all roles. `pg_user` is a view showing only roles with LOGIN privilege.

Key Column	Description
`rolname`	Role name
`rolsuper`	Superuser?
`rolinherit`	Inherits role privileges?
`rolcreaterole`	Can create roles?
`rolcreatedb`	Can create databases?
`rolcanlogin`	Can log in?
`rolconnlimit`	Max connections (-1 = unlimited)
`rolvaliduntil`	Password expiry timestamp

```
SELECT rolname, rolsuper, rolinherit, rolcreaterole, rolcreatedb,
       rolcanlogin, rolconnlimit,
       rolvaliduntil
FROM pg_roles
ORDER BY rolname;
```

Key Views

pg_stat_activity

One row per backend process (including autovacuum, WAL sender, etc.).

Key Column	Description
`pid`	Process ID
`datname`	Database name

`usename`	Role name
`application_name`	`application_name` GUC from the client
`client_addr`	Client IP address
`state`	`active`, `idle`, `idle in transaction`, `fastpath function call`
`query`	Current or last query (truncated to `track_activity_query_size`)
`wait_event_type`	Category of wait: Lock, LWLock, IO, Client, etc.
`wait_event`	Specific wait event name
`query_start`	When the current query started
`xact_start`	When the current transaction started
`backend_start`	When this backend connected
`backend_type`	`client backend`, `autovacuum worker`, `walsender`, etc.

---

pg_stat_user_tables

Cumulative statistics per user table. Updated by the stats collector.

Key columns: `relid`, `schemaname`, `relname`, `n_live_tup`, `n_dead_tup`, `n_mod_since_analyze`, `n_ins_since_vacuum`, `last_vacuum`, `last_autovacuum`, `last_analyze`, `last_autoanalyze`, `vacuum_count`, `autovacuum_count`, `analyze_count`, `autoanalyze_count`.

---

pg_stat_user_indexes

Cumulative statistics per index.

Key columns: `relid`, `indexrelid`, `schemaname`, `relname`, `indexrelname`, `idx_scan`, `idx_tup_read`, `idx_tup_fetch`.

`idx_scan = 0` → index has never been used → candidate for removal.

---

pg_locks

One row per lock currently held or awaited.

Key Column	Description
`pid`	Backend holding or waiting for the lock
`locktype`	`relation`, `tuple`, `transactionid`, `page`, etc.
`relation`	OID of the locked relation (for relation locks)
`classid` + `objid`	For advisory locks
`mode`	Lock mode (`AccessShareLock`, `ExclusiveLock`, etc.)
`granted`	true = holding lock, false = waiting

`transactionid`	XID (for row-level locks)

---

pg_replication_slots

One row per replication slot (logical or physical).

Key Column	Description
`slot_name`	Unique slot name
`plugin`	Logical decoding plugin (for logical slots)
`slot_type`	`physical` or `logical`
`database`	Database name (logical) or NULL (physical)
`active`	Is a process currently connected to this slot?
`confirmed_flush_lsn`	LSN up to which the consumer has confirmed receipt
`restart_lsn`	Oldest WAL still needed by this slot
`wal_status`	`reserved`, `extended`, `unreserved`, `lost`

```
-- Dangerous: idle slot holding WAL
SELECT slot_name, slot_type, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS lag
FROM pg_replication_slots
WHERE NOT active;
```

Catalog Query Patterns

Object lookup by name

```
-- Convert name to OID
SELECT 'orders'::regclass;       -- OID of table 'orders'
SELECT 'orders'::regclass::oid;  -- as plain oid

-- Convert OID back to name
SELECT 16385::regclass;          -- table name for OID 16385
SELECT oid::regclass FROM pg_class WHERE oid = 16385;
```

Finding dropped columns

```
SELECT attname, attnum, atttypid::regtype
FROM pg_attribute
WHERE attrelid = 'orders'::regclass
  AND attisdropped = true;
-- Dropped columns show as pg_dropped_<n>
```

Checking effective permissions

```
-- Does user 'alice' have SELECT on 'orders'?
SELECT has_table_privilege('alice', 'orders', 'SELECT');

-- What privileges does current user have?
SELECT privilege_type
FROM information_schema.role_table_grants
WHERE table_name = 'orders'
  AND grantee = current_user;
```

20+ Useful Catalog Queries

```
-- Q1. List all user tables with size and row count
SELECT n.nspname, c.relname,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size,
       c.reltuples::bigint AS est_rows
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
  AND n.nspname NOT IN ('pg_catalog', 'information_schema', 'pg_toast')
ORDER BY pg_total_relation_size(c.oid) DESC;

-- Q2. Find all indexes on a table with their columns
SELECT i.relname AS index, ix.indisprimary AS pk, ix.indisunique AS uniq,
       string_agg(a.attname, ', ' ORDER BY k.pos) AS columns,
       pg_size_pretty(pg_relation_size(i.oid)) AS index_size
FROM pg_index ix
JOIN pg_class i ON i.oid = ix.indexrelid
JOIN pg_class t ON t.oid = ix.indrelid
JOIN pg_attribute a ON a.attrelid = t.oid
CROSS JOIN LATERAL unnest(ix.indkey) WITH ORDINALITY k(col, pos)
WHERE a.attnum = k.col AND t.relname = 'orders'
GROUP BY i.relname, ix.indisprimary, ix.indisunique, i.oid
ORDER BY i.relname;

-- Q3. Find unused indexes (never scanned)
SELECT schemaname, relname, indexrelname, idx_scan,
pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
WHERE idx_scan = 0
  AND NOT EXISTS (
      SELECT 1 FROM pg_index ix
      WHERE ix.indexrelid = pg_stat_user_indexes.indexrelid
        AND (ix.indisprimary OR ix.indisunique)
  )
ORDER BY pg_relation_size(indexrelid) DESC;

-- Q4. Find long-running queries
```

```sql
SELECT pid, now() - query_start AS duration, state, query
FROM pg_stat_activity
WHERE state != 'idle'
  AND query_start < now() - interval '60 seconds'
ORDER BY duration DESC;

-- Q5. Find blocking locks
SELECT
    blocked.pid         AS blocked_pid,
    blocked.query       AS blocked_query,
    blocking.pid        AS blocking_pid,
    blocking.query      AS blocking_query
FROM pg_stat_activity blocked
JOIN pg_locks         bl ON bl.pid = blocked.pid AND NOT bl.granted
JOIN pg_locks         kl ON kl.transactionid = bl.transactionid AND kl.granted
JOIN pg_stat_activity blocking ON blocking.pid = kl.pid;

-- Q6. Table column metadata
SELECT a.attnum, a.attname, t.typname,
       a.attnotnull, a.atthasdef,
       CASE a.attstorage WHEN 'p' THEN 'PLAIN' WHEN 'e' THEN 'EXTERNAL'
           WHEN 'm' THEN 'MAIN' WHEN 'x' THEN 'EXTENDED' END AS storage,
       a.attstattarget
FROM pg_attribute a
JOIN pg_type t ON t.oid = a.atttypid
WHERE a.attrelid = 'orders'::regclass AND a.attnum > 0 AND NOT a.attisdropped
ORDER BY a.attnum;

-- Q7. Show all foreign keys and their referenced tables
SELECT
    tc.table_name, kcu.column_name,
    ccu.table_name AS foreign_table, ccu.column_name AS foreign_column,
    rc.update_rule, rc.delete_rule
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu ON kcu.constraint_name =
tc.constraint_name
JOIN information_schema.referential_constraints rc ON rc.constraint_name =
tc.constraint_name
JOIN information_schema.constraint_column_usage ccu ON ccu.constraint_name =
rc.unique_constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY'
  AND tc.table_schema = 'public';

-- Q8. Query planner statistics for a column
SELECT attname, null_frac, avg_width, n_distinct,
       array_length(most_common_vals, 1) AS mcv_count,
       correlation
FROM pg_stats
WHERE tablename = 'orders' AND attname = 'status';

-- Q9. Find all functions in a schema
SELECT proname, pg_get_function_arguments(oid) AS args,
```

```sql
       pg_get_function_result(oid) AS returns,
       l.lanname AS language,
       provolatile
FROM pg_proc p
JOIN pg_language l ON l.oid = p.prolang
WHERE pronamespace = (SELECT oid FROM pg_namespace WHERE nspname = 'public')
ORDER BY proname;

-- Q10. List all active connections by user and database
SELECT datname, usename, count(*) AS connections,
       count(*) FILTER (WHERE state = 'active') AS active
FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
GROUP BY datname, usename
ORDER BY connections DESC;

-- Q11. Show tables with the most dead tuples
SELECT relname, n_dead_tup, n_live_tup,
       round(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS dead_pct
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 10;

-- Q12. Find all constraints on a table
SELECT conname, contype,
       CASE contype
           WHEN 'p' THEN 'PRIMARY KEY'
           WHEN 'u' THEN 'UNIQUE'
           WHEN 'f' THEN 'FOREIGN KEY'
           WHEN 'c' THEN 'CHECK'
           WHEN 'x' THEN 'EXCLUSION'
       END AS type,
       pg_get_constraintdef(oid) AS definition
FROM pg_constraint
WHERE conrelid = 'orders'::regclass;

-- Q13. Index usage ratio (scans vs sequential)
SELECT relname,
       seq_scan, idx_scan,
       round(100.0 * idx_scan / NULLIF(seq_scan + idx_scan, 0), 1) AS idx_pct,
       pg_size_pretty(pg_relation_size(relid)) AS table_size
FROM pg_stat_user_tables
WHERE seq_scan + idx_scan > 100
ORDER BY idx_pct ASC;

-- Q14. Find tables with stale statistics
SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
FROM pg_stat_user_tables
WHERE n_mod_since_analyze > 10000
ORDER BY n_mod_since_analyze DESC;

-- Q15. Show replication lag
SELECT application_name, state,
```

```
       sent_lsn, write_lsn, flush_lsn, replay_lsn,
       pg_size_pretty(sent_lsn - replay_lsn) AS total_lag
FROM pg_stat_replication;

-- Q16. Check for idle-in-transaction sessions (dangerous)
SELECT pid, usename, application_name,
       now() - xact_start AS xact_duration, query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
ORDER BY xact_duration DESC;

-- Q17. Table partition information
SELECT p.relname AS partition, n.nspname AS schema,
       pg_get_expr(c.relpartbound, c.oid) AS partition_bound,
       pg_size_pretty(pg_relation_size(c.oid)) AS size
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
JOIN pg_inherits i ON i.inhrelid = c.oid
JOIN pg_class p ON p.oid = i.inhparent
WHERE p.relname = 'orders'
ORDER BY partition;

-- Q18. Show all sequences and their current values
SELECT c.relname, n.nspname,
       pg_sequence_last_value(c.oid) AS last_value,
       s.seqstart, s.seqincrement, s.seqmax, s.seqcycle
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
JOIN pg_sequence s ON s.seqrelid = c.oid
WHERE c.relkind = 'S'
  AND n.nspname = 'public'
ORDER BY c.relname;

-- Q19. Find all advisory locks
SELECT pid, classid, objid, objsubid, mode, granted
FROM pg_locks
WHERE locktype = 'advisory'
ORDER BY pid;

-- Q20. Check pg_hba.conf entries (PG 10+)
SELECT line_number, type, database, user_name, address, auth_method
FROM pg_hba_file_rules
ORDER BY line_number;

-- Q21. Show XID wraparound risk for all databases
SELECT datname, age(datfrozenxid) AS db_age,
       2000000000 - age(datfrozenxid) AS xids_remaining
FROM pg_database
ORDER BY age(datfrozenxid) DESC;

-- Q22. Find bloated indexes (deletion-heavy workloads)
SELECT
```

```
    schemaname, tablename, indexname,
    idx_scan,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE idx_scan < 100
ORDER BY pg_relation_size(indexrelid) DESC;
```

ASCII Diagram: Catalog Relationships

```
pg_namespace (schemas)
    │
    ├── pg_class (tables, indexes, views…)
    │       │
    │       ├── pg_attribute (columns per relation)
    │       │       └── pg_type (data types)
    │       │
    │       ├── pg_index (index metadata per index)
    │       │
    │       ├── pg_constraint (pk, fk, check, unique)
    │       │
    │       └── pg_statistic / pg_stats (column statistics)
    │
    ├── pg_proc (functions/procedures)
    │
    └── pg_trigger (triggers)

pg_roles / pg_authid (cluster-wide)
    └── pg_auth_members (role membership)

pg_database (cluster-wide)
    └── pg_tablespace (storage locations)

MONITORING VIEWS (read-only, dynamic):
  pg_stat_activity         ← live backend sessions
  pg_stat_user_tables      ← per-table I/O and vacuum stats
  pg_stat_user_indexes     ← per-index usage stats
  pg_stat_bgwriter         ← checkpoint / bgwriter stats
  pg_stat_replication      ← streaming replication lag
  pg_locks                 ← current lock state
  pg_replication_slots     ← replication slot metadata
  pg_stat_progress_vacuum  ← live VACUUM progress
  pg_stat_wal              ← WAL generation stats (PG 15+)
```

Common Mistakes

1. **Forgetting to filter by schema.** `SELECT * FROM pg_class WHERE relname = 'users'` may return rows from multiple schemas. Always join `pg_namespace` or use `relnamespace = 'public'::regnamespace`.

2. **Using `pg_class.reltuples` as exact row count.** `reltuples` is an estimate updated only by VACUUM/ANALYZE. For exact counts, use `SELECT count(*) FROM table`.
3. **Not filtering out dropped columns.** `pg_attribute` contains ghost rows for dropped columns (with `attisdropped = true`). Always add `AND NOT attisdropped` when listing columns.
4. **Confusing catalog tables with statistics views.** Catalog tables (`pg_class`, `pg_attribute`) are always current. Statistics views (`pg_stat_user_tables`, `pg_statistic`) are updated asynchronously by the statistics collector and may be slightly stale.
5. **Assuming `idx_scan = 0` always means an index is unused.** Statistics reset with `pg_stat_reset()` or after a server restart. Check the server start time and when statistics were last reset.
6. **Querying `pg_statistic` directly.** It contains binary encoded statistics. Use the `pg_stats` view which decodes them into readable form.

Best Practices

- **Use `regclass`, `regtype`, `regproc`** casts for human-readable OID resolution.
- **Always filter `attisdropped = false`** when querying `pg_attribute`.
- **Use `information_schema` views** when portability across SQL dialects matters; use `pg_catalog` for PostgreSQL-specific detail.
- **Schedule `pg_stat_reset_shared()`** resets carefully — they wipe cumulative counters, breaking trend analysis.
- **Build a catalog query library** for your team: common queries for unused indexes, bloated tables, lock waits, etc.
- **Monitor `pg_replication_slots`** — an inactive slot holding WAL can fill `pg_wal/` and crash the server.

Performance Considerations

- **Catalog queries are generally fast** because catalog tables are usually small and well-indexed. However, on systems with thousands of tables/indexes/functions, complex catalog queries can take seconds.
- **`pg_stat_user_tables` queries should be quick** but `pg_relation_size()` requires filesystem stat calls — avoid calling it for thousands of tables in a loop.
- **`information_schema` views can be slow** because they join many catalog tables. Prefer `pg_catalog` queries for performance-critical monitoring.
- **`EXPLAIN`-ing catalog queries** can reveal unexpected sequential scans on catalog tables — always check if statistics are up to date.

Interview Questions

Q1. What is `pg_class` and what does `relkind` distinguish?

> *`pg_class` is the central catalog table for all relations. `relkind` identifies the type: `r` =ordinary table, `i` =index, `S` =sequence, `v` =view, `m` =materialized view, `c` =composite type, `t` =TOAST table, `p` =partitioned table, `f` =foreign table.*

Q2. When would `pg_class.relfilenode` differ from `pg_class.oid`?

> *After any operation that physically rewrites the relation file: `VACUUM FULL`, `CLUSTER`, `TRUNCATE`, or `ALTER TABLE ... SET TABLESPACE`. The OID (logical identity) stays the same but the file on disk gets a new name.*

Q3. How would you find which indexes on a table have never been used?

> *Query `pg_stat_user_indexes WHERE idx_scan = 0` and exclude primary key and unique indexes (which are used for constraint enforcement regardless of scan count).*

Q4. What does `n_distinct` in `pg_stats` mean, and what does a negative value indicate?

> *`n_distinct` is the estimated number of distinct values. A positive value is the absolute count. A negative value is a fraction of the total row count — e.g., `-0.05` means the column has approximately `0.05 × row_count` distinct values. This encoding is used when the distinct count scales with table size (e.g., a mostly-unique column).*

Q5. Why is `pg_stat_user_tables.n_dead_tup` an estimate?

> *It is updated by the statistics collector, which processes stats asynchronously. The actual dead tuple count on disk may differ by a few hundred to a few thousand tuples from what the view shows.*

Q6. What is the danger of an inactive `pg_replication_slots` entry?

> *An inactive replication slot still holds its `restart_lsn`, preventing PostgreSQL from recycling WAL segments older than that LSN. Over time, this can cause `pg_wal/` to fill the disk completely, crashing the server.*

Q7. How do you find blocking queries and the queries they block?

> *Join `pg_stat_activity` with `pg_locks` on `pid`, filtering for `NOT granted` (waiting) and `granted` (holding) rows and matching on `transactionid` or `relation`.*

Q8. What is the difference between `pg_roles` and `pg_user`?

> *`pg_roles` lists all roles (including groups that cannot log in). `pg_user` is a view that filters to only roles with `rolcanlogin = true`. `pg_roles` is cluster-wide (in `global/`); both are accessible from any database.*

Exercises with Solutions

Exercise 1 — Schema explorer

```
-- Solution: list every user table with its column count, index count, and total
size
SELECT
    n.nspname AS schema,
    c.relname AS table_name,
    (SELECT count(*) FROM pg_attribute a
     WHERE a.attrelid = c.oid AND a.attnum > 0 AND NOT a.attisdropped) AS columns,
    (SELECT count(*) FROM pg_index ix WHERE ix.indrelid = c.oid) AS indexes,
    pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size,
    c.reltuples::bigint AS est_rows
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
```

```
WHERE c.relkind = 'r'
  AND n.nspname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(c.oid) DESC;
```

Exercise 2 — Idle-in-transaction cleanup query

```
-- Solution: identify sessions that should be killed
SELECT pid, usename, application_name, client_addr,
       now() - xact_start AS idle_duration,
       state, query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
  AND now() - xact_start > interval '5 minutes'
ORDER BY idle_duration DESC;

-- To terminate them:
-- SELECT pg_terminate_backend(pid)
-- FROM pg_stat_activity
-- WHERE state = 'idle in transaction'
--   AND now() - xact_start > interval '5 minutes';
```

Exercise 3 — Full index audit

```
-- Solution: index audit with usage, size, bloat indicators
SELECT
    n.nspname AS schema,
    t.relname AS table,
    i.relname AS index,
    ix.indisprimary AS pk,
    ix.indisunique  AS unique,
    s.idx_scan,
    pg_size_pretty(pg_relation_size(i.oid)) AS index_size,
    CASE WHEN s.idx_scan = 0 AND NOT ix.indisprimary AND NOT ix.indisunique
         THEN 'CANDIDATE FOR REMOVAL'
         ELSE 'in use'
    END AS status
FROM pg_index ix
JOIN pg_class i ON i.oid = ix.indexrelid
JOIN pg_class t ON t.oid = ix.indrelid
JOIN pg_namespace n ON n.oid = t.relnamespace
LEFT JOIN pg_stat_user_indexes s ON s.indexrelid = ix.indexrelid
WHERE n.nspname = 'public'
ORDER BY pg_relation_size(i.oid) DESC;
```

Cross-References

- **01_storage_architecture.md** — `pg_class.relfilenode`, `pg_tablespace`
- **02_heap_storage.md** — `pg_class.relpages`, `reltuples`

- **06_vacuum.md** — `pg_stat_user_tables.n_dead_tup`, `last_autovacuum`
- **07_autovacuum.md** — Autovacuum settings stored in `pg_class.reloptions`
- **11_query_execution_pipeline.md** — Planner uses `pg_statistic` for cost estimates